UNIT-01

Ajanta
Page No.
Date

Introduction to Java

Java is a high level, robust, object-oriented and secured programming language developed by James Gosling at Sun microsystems Inc in year 1995 and later acquired by Oracle Corporation.

Features of Java / Java Buzzwords

* Object oriented. (Based on classes and objects)
* Platform independent (Compiler converts source code to bytecode & then JVM executes the bytecode generated by the compiler.)
* Portable (Easy to move b/w platforms)
* Distributed (Supports network based applications)
* Robust (Both Java compiler & interpreter provide extensive error checking, Automatic garbage collection)
* Secure (Provides several runtime security mechanisms).
* High-performance (JIT improves execution speed)
* Multithreaded (Supports concurrent execution)
* Dynamic (Class can be loaded at runtime)
* Simple. (Easy to learn & use).

Hello World Program

```
public class HelloWorld {
    public static void main (String[] args) {
        System.out.println("Hello World");
    }
}
```

Add Two Numbers

```
public class AddNum {
    public static void main (String[] args) {
        int a = 10;
        int b = 20;
        int sum = a+b;

        System.out.println("Sum=" + sum);
    }
}
```

Even or Odd

```
public class EvenOdd {
    public static void main (String[] args) {
        int num = 15;

        if (num % 2 == 0) {
            System.out.println("Even");
        }
        else {
            System.out.println("odd");
        }
    }
}
```

for loop

```
public class forloopExample {
    public static void main (String[] args) {
        for (int i=1 ; i<=5 ; i++){
            system.out.println(i);
        }
    }
}
```

Find largest Number

```
public class largest Number {
    public static void main (String[] args){
        int a=15, b=25, c=10;

        int largest=a;

        if (b>largest) largest=b;
        if (c>largest) largest=c;

        system.out.println("largest= " + largest);
    }
}
```

* write a program to print sum of series:

$1+2+3+\ldots+n \longrightarrow n(n+1)/2$

$1^2+2^2+3^2+\ldots+n^2 \longrightarrow \frac{n(n+1)(2n+1)}{6}$

* WAP to print LCM & GCD of two numbers.

Page No.
Date

```
pubic class lcmhcf {
    public static void main (String [] args) {

        int a=10;
        int b=15;
        int hcf=1;

        for (int i=2 ; i<=a ; i++)

            if ((a%i==0)&& (b%i==0)) {
                hcf=i;
            }

        System.out.println (hcf);

        int lcm= ((a*b)/hcf);
        System.out.println (lcm);

    }
}
```

Java Programming Environment and Runtime Environment

Java has two main environments that are important for developing and running Java programs.

(i) Java Programming Environment

(ii) Java Runtime Environment (JRE).

Java Programming Environment

It is the setup used to write, compile, debug and develop Java programs.

The main component is the Java Development Kit (JDK)

* Main Components of JDK:-

(i) Java Development Kit (JDK):- Complete package required for Java Development.

(ii) Java Runtime Environment (JRE):- Provides the environment needed to run Java programs.

(iii) Java Virtual Machine (JVM):- Executes Java bytecode.

(iv) Java Compiler (javac):- Converts Java source code (.java) into bytecode (.class).

(v) Java Debugger:- Helps find and fix errors in program.

(vi) Java Documentation tool:- Used to create documentation from java source code.

How Java Program

Java Source code
↓
javac compiler
↓
Bytecode (.class file)
↓
JVM
↓
Machine Code
↓
Program output

Java Runtime Environment (JRE)

It provides everything required to run a Java application, but it is not intended for developing or compiling Java programs.

The JRE mainly consists of:-

(i) Java virtual machine (JVM):- It is responsible for executing Java bytecode.

* It provide Java's famous platform independence.

(ii) Java class libraries:- Java provides many predefined classes and packages such as:
- java.lang
- java.util
- java.io
- java.net
- java.awt

(iii) Supporting Runtime files.

Java Development Platforms

A Java Development platform is a software environment that provides the tools and libraries needed to develop, compile, test, debug and run Java applications.

Types of Java Development Platforms

(i) Java Standard Edition (Java SE) :-

Java SE is the basic and most commonly used Java platform. It is used to develop general purpose applications.

Used for :-
* Desktop applications
* Basic Java programs
* Core Java programming
* Learning Java fundamentals.

(ii) Jakarta Enterprise Edition (Jakarta EE) :-

It is designed for developing large-scale enterprise and web applications.
It provides APIs and services for building applications that run on application servers.

Used for :-
* Web applications
* Enterprise applications
* Server-side applications
* Distributed system
* Large business applications.

(iii) Java Micro Edition (Java ME):-

Ajanta
Page No.
Date

* It is used for applications running on resource-constrained devices.

Used for:
* Embedded devices
* Small electronic devices
* IoT related systems
* older mobile and connected devices.

Byte Code in Java

* It is an intermediate code generated by the Java compiler after compiling a Java source program.
* It is stored in a file with the .class extension.

Features of Bytecode:-

* Platform Independent
* Compact
* Portable
* Secure.

Java applet:-

* It is a small Java program that was designed to run inside a web browser or an applet viewer.
* Does not normally have a main() method.
* Uses predifined applet lifecycle methods.
* Provides security restrictions through the Java sandbox.

Applet life Cycle

Ajanta
Page No.
Date

The main methods of an applet are :-

(i) init() :- Called when applet is initialized.

(ii) start() :- Called when applet starts or becomes active

(iii) paint() :- Used to display graphics or text.

(iv) stop() :- Called when the applet becomes inactive

(v) destroy() :- Called before applet is removed from memory.

Java Program Structure

```
class Name {
    public static void main ( String [] args) {
        System.out.println (" - - - ");
    }
}
```

public :- accessible to Jvm

static :- can be called without creating an object

void or :- does not return a value.

main :- Method name recognized as the application entry point

String [] args :- receives - command line arguments.

Comments :-

// This is a single line comment

/* This is a
multi-line comment
*/

Garbage Collection in Java

* It is an automatic memory management process in Java.

* It automatically finds objects that are no longer being used by the program and removes them from memory.

How garbage collection works :-

Objects created
↓
Objects stored in Heap
↓
JVM identifies unreachable objects
↓
Garbage Collector
↓
Memory Reclaimed.

Heap : The heap is the memory area where Java objects are generally stored.

Ex- class Demo{

```
public static void main(String[] args){
    Demo obj1 = new Demo();
    Demo obj2 = new Demo();

    obj1 = null;
    obj2 = null;

    System.gc();
  }
}
```

Advantages of Garbage Collection

* Automatic memory management
* Reduces memory leaks
* Improves programming productivity.
* Helps manage heap memory.
* Provides safer memory handling.

Lexical Issues in Java

* Lexical issues are the rules related to how a Java program is broken down into tokens by the compiler.
* In simple words they describe the basic elements that made up the Java source code.
* The Java compiler reads the source code and identifies elements such as keywords, identifiers, literals, operators, separators and comments.

Primitive Data Types in Java

	Data Type	Size (Bytes)	Range / Values
integer	Byte	1 byte = 8 bits	-128 to 127
	short	2 bytes	-32768 to 32767
	int	4 bytes	-2^{31} to $2^{31}-1$
	long	8 bytes	-2^{63} to $2^{63}-1$
floating point	float	4 bytes	$\pm 3.4 \times 10^{38}$
	double	8 bytes	$\pm 1.7 \times 10^{308}$
	char	2 bytes	0 to 65,535
	Boolean	not specified	true / false

Type Conversion :-

* Type conversion is the process of converting one data type to another automatically by Java.

* It usually happens when a smaller data type is converted into a larger compatible data type.

* byte → short → int → long → float → double
  char → int

Ex:-

```
int x = 25;
long y = x;
float z = y;
```

Type Casting :-

* It is the process of converting a value from one data type to another. manually using the casting Operator (type).

* It mainly required when converting a larger data type into smaller data type.

Ex:-
```
double x = 10.75;
int y = (int) x;
System.out.println(y);
```

* double → float → long → int → short → byte.

Ex:-
```
public class Conversion {
    public static void main (String[] args) {

        // Type Conversion.
        int a = 20;
        double b = a;

        // Type Casting.
        double y = 20.75;
        int y = (int) x;

        System.out.println(b);

        System.out.println(y);
    }
}
```

Variables in Java:-

A variable is a named memory location used to store data that can change during the execution of a program.

(i) **Local variable**:- A variable declared inside a method or block is called a local variable.

```
class Demo {
    public static void main(String[] args){
        int marks = 90;
        System.out.println(marks);
    }
}
```

(ii) **Instance Variable**:- An instance variable belongs to an object.

```
class Student {
    String name;
    int age;
}
```

(iii) **Static Variable**:- A static variable belongs to the class instead of individual objects.

```
class Student {
    static String college = "ABC college";
    String name;
}
```

Variable Declaration + Initialization.

```
int age;
age = 20;
```

Ex:-
```
class Demo {
    public static void main (String[] args) {
        int age=20; // local variable.
        System.out.println("Age:" + age);
    }
}
```
Output:- Age: 20

Ex:-
```
class Student {
    String name; // instance variable
    int age;  // instance variable.

    void display() {
        System.out.println("Name=" + name);
        System.out.println("Age=" + age);
    }

    public static void main (String[] args) {
        Student s1= new Student();
        Student s2= new Student();
        s1.name="Rahul";        s2.name="Mahesh";
        s1.age= 20;             s2.age=21;

        s1.display();           s2.display();
    }
}
```

Output:-
```
Name = Rahul
Age = 20
Name = Mahesh
Age = 21
```

Ex:-

```
class student{
    string name;
    static string college="ABC college"; // static variable

    void display(){
        System.out.println(name + " - " + college);
    }
public static void main(String[] args)
        student s1= new student();
        student s2= new student();

        s1.name="Rahul";
        s2.name="Aman";

        s1.display();
        s2.display();
    }
}
```

Output:-

Rahul - ABC college
Aman - ABC college.

Ex:-

```
class C
{
  public int x; // instance variable

  public void show(){
     int z=20; // local variable.
     System.out.println(z);
    }
public class main{
   public static void main(String[] args){
        C obj = new C();
        obj.x=15;
        obj.show(); // prints 20
        System.out.println(obj.x); // prints 15
       }
}
```

Ex:-
```
class C {
    public int x;
    static int y=10; // static variable
}
public class Main {
    public static void main (String[] args) {
        C obj = new C();
        C obj1 = new C();

        obj.y=15;
        obj1.y=25;

        System.out.println(obj.y); // prints 25
        System.out.println(obj1.y); // print 25
    }
}
```

{ Since y is static variable it remains same for every object }

Ex:-
```
class C {
    public int x;
    static int y=10;
}
public class Main {
    public static void main (String[] args) {
        C obj = new C();
        C obj1 = new C();

        obj.x=5;
        obj1.x=6;

        System.out.println(obj.y); // Prints 10
        System.out.println(obj1.y); // Prints 10
        System.out.println(obj.x); // Prints 5
        System.out.println(obj1.x); // Prints 6.
    }
}
```

Strings in Java

Ajanta
Page No.
Date

* A string is a sequence of characters used to represent and manipulate text.
* String is not a primitive data type. It is in class java.lang package.
* Strings are immutable and their indexing starts from
* String supports unicode characters.
* Strings literals are stored in string pool.

Ways to Create a String

(i) **Using String literal**:- Most commonly used method. When a string literal is created, Java can store it in a special memory called string Pool.

Ex: `String str = "Hello";`

(ii) **Using new keyword**:- A string can also be created using new keyword.

Ex: `String str = new String("Hello");`

Ex: **Program to Print a String**

```
class StringDemo {
    public static void main(String[] args) {
        String str = "Hello";
        System.out.println(str);
    }
}
```

String Methods

Method	Description
length()	Returns length
charAt()	Returns character at the given index
equals()	Compares contents
equalsIgnoreCase()	Compares ignoring case
toUpperCase()	Converts to upper case
toLowerCase()	Converts to lower case
substring()	Extracts part of string
concat()	Joins string
contains()	checks for sequence
indexOf()	finds index
replace()	Replace char/text
trim()	Removes surrounding whitespace
isEmpty()	Checks if empty
startsWith()	Checks beginning
endsWith()	Checks ending.

Ex:-

```
import java.util.Scanner;
class StringDemo{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a string:");
        String str = sc.nextLine();
    System.out.println("String:" + str);
        "    "    "  ("Length:" + str.length());
        "    "    "  ("Upper Case:" + str.toUpperCase());
        "    "    "  ("Lower Case:" + str.toLowerCase());
    if (!str.isEmpty()){
        System.out.println("First Character" + str.charAt(0));
        }
        System.out.println("Contains 'Java':" + str.contains
                                        ("Java"));
    }
}
```

Ex:- Count vowels in a string

```
import java.util.Scanner;

class Vowels {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter string:");

        String str = sc.nextLine().toLowerCase();

        int countv = 0;
        int countc = 0;
        for (int i=0; i< str.length(); i++) {

            char ch = str.charAt(i);
        if (ch>='a' && ch<='z') {
            if (ch=='a' || ch=='e' || ch=='i' || ch=='o' ||
                                        ch=='u') {

                countv++;
            } ------> else {
        } <------ countc++;
        }}
            System.out.println("No. of vowels: " + countv);
        } System.out.println("No. of consonants: " + countc);
    }
```

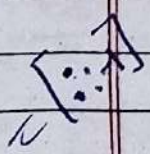
Ex:- Reverse a string and check Palindrom

```
import java.util.Scanner;
class RevString {
    public static void main (String[] args) {
        Scanner sc= new Scanner (System.in);
        System.out.println("Enter a string:");
        String str= sc.nextLine();
        String rev="";
        for (int i= str.length()-1 ; i>=0 ; i--) {
            rev= str.charAt(i);
        }
        System.out.println("Reverse" + rev);
        if (str.equals(rev)) {
            System.out.println("Palindrome");
        }
        else{
            System.out.println("Not Palindrome");
        }
    }
}
```

Reverse the string.

Ex:- Count Frequency of a character.

```
import java.util.Scanner;

class CharacterFreq {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter string");
        String str = sc.nextLine();

        System.out.print("Enter character");
        char ch = sc.next().charAt(0);

        int count = 0;

        for (int i = 0; i < str.length(); i++) {

            if (str.charAt(i) == ch) {
                count++;
            }
        }

        System.out.println("Frequency=" + count);
    }
}
```



Ex: Count Digits, Letters and special characters

Ajanta
Page No.
Date

```
import java.util.Scanner;
class CharTypes {
    public static void main(String[] args){
        String str = "Hello123$Ajay";
    int l=0, d=0, s=0;
for(int i=0; i<str.length(); i++){
    char ch= str.charAt(i);
    if(Character.isLetter(ch)){
        l++;
    }
    else if(Character.isDigit(ch)){
        d++;
    }
    else{
        s++;
    }
}
System.out.println("Letters="+l);
    "   "   "  ("Digit="+d);
    "   "   "  ("Special characters="+s);
```

Arrays in Java

* An array is a fixed size collection of elements of the same data type, where each element is accessed using index.
* Arrays store multiple values under one variable name.
* The size of array is fixed after creation.
* Arrays are objects in Java.
* Arrays can be one dimensional or multidimensional.

Declaring an Array:-

(i) `int[] arr;` (ii) `int arr[];`

Creating an Array:-

```
int [] arr = new int[5];
```

Initializing an Array:-

(i)
```
int[] arr = new int [5];
arr[] = {10, 20, 30, 40, 50};
```

(ii)
```
int [] arr = new int [5];
arr[0] = 1;
arr[1] = 2;
:
```

Ex:
```
int [] marks = {80, 85, 90};
System.out.println (marks[0]); ⟶ 80
                   (marks[1]); ⟶ 85
                   (marks[2]); ⟶ 90
```

Ex: Taking Array Input from user.

```
import java.util.Scanner;
public static void main(String[] args){

    Scanner sc = new Scanner(System.in);

    System.out.println("Enter size:");
     int n = sc.nextInt();
     int[] arr = new int[n];
    System.out.println("Enter elements");

    for(int i=0; i<n; i++){
        arr[i] = sc.nextInt();
        }

    System.out.println("Array elements");
        for(int i=0; i<n; i++){
            System.out.println(arr[i]);
        }
    }
}
```

Ex: Sum of elements of array.

```
class ArraySum {
    public static void main(String[] args){
    int arr[]={10, 20, 30, 40, 50};
     int sum=0;
    for(int x: arr){
        sum+=x;
     }
    System.out.println("Sum="+sum);
    }
}
```

Ex.1 Find largest and smallest element in Array

```
class large and small {
public static void main (String[] args) {

    int arr[]= {25, 10, 45, 30, 15};

    int max= arr[0];
    int min= arr[0];
for (int i=1; i<arr.length; i++){
    if (arr[i] > max)
        max= arr[i];
    }
}

for (int i=1; i<arr.length; i++) {
  if (arr[i] < min){
     min= arr[i];
  }
}

System.out.println(" Max element= " + max);

System.out.println(" Min element= " + min);
```

Ex: search element in an Array

```
class Search {
public static void main (String[] args) {

int[] arr = {10, 20, 30, 40, 50};
  int target = 30;

  int index = -1;

for (int i=0; i< arr.length ; i++) {

  if (arr[i] == target ) {
      index = i;
      break;
  }
}
 if (index != -1)
  System.out.println("Element found at index" +index);

else
  System.out.println("Element not found");
 }
}
```

Difference b/w Strings and Arrays

Array	Strings
* An array stores multiple values of same data type.	* A string is a sequence of characters / text.
* Size is fixed after creation.	* String objects are immutable.
* Elements are accessed using index.	* Characters can be accessed using charAt().
* Uses length property.	* Uses length() method.
* Arrays are object in java.	* Strings are class in Java.

Ex: Find second largest element in an Array.

```
public class SecondLargest{
  public static void main ( String[] args){
    int [] arr= {10, 25, 35, 40, 42 };

int largest= Integer.MIN_VALUE; //Smallest possible int value
int secondlargest= Integer.MIN_VALUE;          in java.

for(int i=0 ; i< arr.length ; i++){
    if (arr[i] > largest){
          secondlargest= largest;
            largest= a[i];
        }
  else if (arr[i] > secondlargest && arr[i] != largest){
            secondlargest= a[i];
          }
  System.out.println("second largest =" + secondlargest)
  }
}
```

Ex:- Reverse an array without using second array

```
public class ReverseArray{
    public static void main(String[] args){

        int[] arr = {10, 20, 30, 40, 50};

        int start = 0;
        int end = arr.length - 1;

        while (start < end) {

            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;          } swap elements.

            start++;
            end--;
        }

        System.out.println("Reversed Array:");

        for(int i=0; i<arr.length; i++){
            System.out.println(arr[i] + " ");
        }

    }
}
```

Vectors in Java

* Vector is a class used to store a collection of elements dynamically. It is similar to an array, but its size can grow or sink automatically.

* Vector belongs to java.util package.

Important vector methods

add() → Adds an element

add(index, element) → Add at a specific index

get(index) → Gets an element

set(index, element) → Updates an element

remove(index) → Removes an element by index

size() → Returns no. of element

isEmpty() → Checks whether Vector is empty.

contains() → Checks whether element exists

indexOf() → Find index of an element

clear() → Removes all elements.

firstElement() → Returns first elements

lastElement() → Returns last element

Capacity() → Returns Capacity.

elementAt(index) → Gets an element

Ex: Creating a vector

```
import java.util.Vector;
public class Main {
    public static void main(String[] args){

        Vector<Integer> v= new Vector<>();

        v.add(10);
        v.add(20);
        v.add(30);

        System.out.println(v);

    System.out.println("Element at index 0="+v.elementAt(0));

        v.clear();
    System.out.println(v);
    }
}
```

Ex: Vector Methods

```
import java.util.Vector;
import java.util.Scanner;

public class VectorExample{
    public static void main(String[]args){

        Scanner sc= new Scanner(System.in);

    Vector<Integer> v= new Vector<>();
```

```
v.add(10);
v.add(20);
v.add(30);
v.add(40);

System.out.println("original vector:"+v);

v.add(2,25);
System.out.println("New vector:"+v);

System.out.println("Element at index 1:"+v.get(1));

v.set(1,50);
System.out.println("After updating index 1:"+v);

System.out.println("Size of vector:"+v.size());

v.remove(2);
System.out.println("After removing index 2:"+v);

System.out.println("Capacity:"+v.capacity());
System.out.println("Elements using loop");
for(int i=0; i<v.size(); i++){
    System.out.println(v.get(i))
}

System.out.println("Elements using enhanced for loop:");

    for(int x:v){
        System.out.println(x);
    }

    v.clear();
    sc.close();
}
```

Operators in Java

Types	Operators
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <, >=, <=
Logical	&&, ||
Assignment	=, +=, -=, *=, /=, %=
Unary	++, --, +, -, !
Bitwise	&, |
Shift	<<, >>, >>>
Ternary	?:

Control statements :- Control statements control the flow or execution order of a Java program. They are mainly :-

(i) Selection statements :- Used when we need to choose one block of code based on a condition.

* if
* if-else
* if-else if
* switch.

(ii) Iteration statements

Iteration statements are also called looping statements. They are used to execute a block repeatedly.

Loop	Condition checked	Minimum execution
for	Before	0 times
while	Before	0 times
do-while	After	1 times

(iii) Jump statements

Used to change the normal flow of execution.

* break :- Terminates the loop or switch immediately.
* continue :- Skips the current iteration and moves to next iteration.
* return :- Used to exit from a method and optionally return a value.

UNIT-02

Ajanta
Page No.
Date

Object-Oriented Programming in Java

It is a programming approach in which programs are designed using classes and objects.

Classes in Java

A class is a blueprint or template for creating objects. It contains data members (variables) and methods (functions) that define the properties and behaviours of objects.

Objects in Java

An object is an instance of a class. It represents a real-world entity and contains data and methods.

Components of a class

(i) **Data members**:- variables declared inside a class.

```
class Student {
    int age;
    String name;
}
```

(ii) **Methods**:- Define the behaviours of an object.

```
class Student {
    void study() {
        System.out.println("Students are studying");
    }
}
```

(iii) **Constructors** :- A constructor is a special member of a class used to initialize objects.

Page No. _____
Date _____

```
class Student {
    String name;

    Student (String n) {
        name = n;
    }
}
```

(iv) **Object** :- It is a instance of a class.

```
Student s1 = new Student ();
Student s2 = new Student ();
```

NOTE :-

* The dot (.) operator is used to access variables and methods of an object.

this keyword

Use to refer to the current object.

```
class Student {
    String name;

    Student (String name) {
        this.name = name;
    }
}
```

this.name → Refers to class variable.

name → Refers to the constructor parameter.

Create a class and object to display student details

```
class Student {
    String name;
    int age;

    void display(){
        System.out.println("Name:" + name);
        System.out.println("Age:" + age);
    }

}
public class Main {
    public static void main(String[] args) {
        Student st = new Student();

        st.name = "Ajay";
        st.age = 20;
        st.display();
    }
}
```

Create multiple objects of a class

```
class Student {
    String name;
    int marks;

    void display(){
        System.out.println(name + " " + marks);
    }
}
```

```
public class Main {
  public static void main(String[] args){
    Student s1= new Student();
    Student s2= new Student();

    s1.name= "Ajay";
    s1.marks = 100;

    s2.name= "Raj";
    s2.marks = 99;

    s1.display();
    s2.display();
  }
}
```

Create a class to Calculate sum of two numbers

```
class Calculator {
  int a,b;

  void sum(){
  System.out.println(" sum=" +(a+b));
  }
}
public class Main {
  public static void main(String[] args){
    Calculator c= new Calculator();
    c.a=10;
    c.b=20;
    c.sum();

  }
}
```

Constructors in Java

Ajanta
Page No.
Date

* It is a special member of class that is used to initialize object.
* Constructor name must be same as the class name.
* It has no return type not even void.
* It is automatically called when an object is created.

Types of Constructors in Java

(1) Default Constructor: If you do not write any constructor, Java automatically provides a default constructor.

```
class Student {
    string name;
    int age;
}
public class Main {
    public static void main (String[] args) {
        Student s = new Student();
        s.name = "Ajay"; s.age = 20;
        System.out.println(s.name);
        System.out.println(s.age);
    }
}
```

(ii) No argument Constructor :- A constructor written by the programmer that takes no arguments is called a non-argument Constructor.

```
class Student {
    String name;
    int age;
    Student() {
    name = "Ajay";
    age = 20;
    }
    void display() {
    System.out.println("Name = " + name + "  " + "Age = "
                                                + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

(iii) Parameterised Constructor :- A constructor that accepts parameters is called parameterized constructor.

It allows us to initialize different objects with different values.

```
class Student {
    String name;
    int age;

    Student (String n, int a) {
        name = n;
        age = a;
    }

    void display () {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

public class Main {
    public static void main (String [] args) {
        Student s1 = new Student ("Ajay", 20);
        Student s2 = new Student ("Ankit", 19);

        s1.display();
        s2.display();
    }
}
```

(iv) Copy constructor :- Java does not provide built in copy constructor like C++, but we can create one ourselves.

A copy constructor initializes one object using another object's value.

```
class Car {
    String brand;
    int price;

    Car (String brand, int price){
        this.brand = brand;
        this.price = price;
    }

    Car (Car c) {
        this.brand = brand;
        this.price = price;
    }

    void display (){
        System.out.println(brand + " " + Price);
    }
}

public class Main {
    public static void main (String [] args){

        Car c1 = new Car ("BMW", 500000);
        Car c2 = new Car (c1);

        c1.display();
        c2.display();
    }
}
```

Method overloading in Java

Method overloading means having more than one method with the same name but different parameters in the same class.

* Compile time - polymorphism.

Ex: Program to calculate area of square, rectangle and triangle

```
class Area {

    void area(int side){
        System.out.println("Area of Square=" +(side*side));
    }

    void area(int length, int breadth){
        System.out.println("Area of Rec=" +(length*breadth));
    }

    void area(double base, double height){
        System.out.println("Area of Triangle=" +(0.5*base*height));
    }
}

public class Main {
    public static void main(String[] args){
        Area a= new Area();

        a.area(10);
        a.area(10,5);
        a.area(2.5, 3.0);

    }
}
```

Using object as Parameters

In java object can be passed as parameter to methods. This is useful when a method needs to work with the data of another object.

Ex:-
```
class Box {
    int length;

    Box (int length) {
        this.length = length;
    }

    void compare (box b) {
        if (length > b.length)
            System.out.println (" first box is biggr ");

        else
            System.out.println (" second box is biggr ");
    }
}

public class Main {
    public static void main (String [] args) {

        Box b1 = new Box(10);
        Box b2 = new Box (20);

        b1.compare (b2);
    }
}
```

Returning objects in Java

In Java, a method can return an object just like it can return an int, double or string.

To return a object, the return type of the method is the class name.

Ex:-

```
class Box {
    int length;

    Box (int l) {
        length = l;
    }

    Box add (Box b) {
// Box result = new Box (length + b.length);
        return result; // return type.
    }
}
public class Main {
    public static void main (String[] args) {
        Box b1 = new box (10);
        Box b2 = new box (20);

        Box b3 = b1.add (b2);

        System.out.println ("Length =" + b3.length);
    }
}
```

Recursion in Java

Recursion is a process in which a method calls itself repeatedly until a specific condition is met.

A recursive method has two important parts:-

(i) Base Condition:- stops the recursion.

(ii) Recursive Call:- method calls itself with a simpler problem.

Ex:- Factorial using Recursion

```
class Factorial {
    int fact(int n){
        if (n==0 || n==1){
            return 1;
        }
        return n* fact(n-1);
    }
}

public class Main{
    public static void main(String[] args){
        Factorial f= new Factorial();
        System.out.println(f.fact(5));
    }
}
```

Ex:- Fibonacci series using Recursion.

Ajanta
Page No.
Date

```
class fibonacci{

    int fib(int n) {
        if (n <= 1) {
            return n;
        }

        return fib(n-1) + fib(n-2);
    }
}

public class main{
    public static void main (String[] args){

        fibonacci f= new fibonacci();

        for (int i=0 ; i<7 ; i++){

            System.out.println(" " + f.fib(i) + " ");
        }
    }
}
```

* for sum of n natural no.s :-

return n + calculate(n-1).

Ex:- Reverse a number using Recursion

```
class Reverse{
    int reverse=0;

    void ReverseNo(int n){
        if (n==0){
            return n;
        }

        reverse= reverse *10 + n%10;
        ReverseNo(n/10);
    }
}

public class Main{
    public static void main(String[] args){

        Reverse r= new Reverse();

        r.ReverseNo(1234);

        System.out.println("Reverse" + r.reverse );
    }
```

Ex:- Power of a Number Using Recursion

```
class Power{
    int Calculate(int base, int exponent){
        if (exponent==0){
            return 1;
        }
        return base * Calculate(base, expment-1);
    }
}
```

```
public class Main {
    public static void main (String [] args) {
        Power p = new Power();
        System.out.println("Result=" + p.calculate(2,5));
    }
}
```

Access Control in Java

Access control means controlling the visibility and accessibility of classes, variables, methods and constructors in Java.

Java provides 4 access modifiers:-

(1) **Default**:- When no access modifier is specified, it is called default / package-private access.

It can be accessed within the same package.

Ex:
```
class Employee {
    int salary = 30000;
}
public class Main {
    public static void main (String [] args) {
        Employee e = new Employee();
        System.out.println(e.salary);
    }
}
```

(ii) **private**:- A private member can only be accessed inside the same class.

Ex:
```
class Account {
    private int balance = 50000;

    void display(){
        System.out.println(balance);
    }
}

public class Main{
    public static void main(String[] args){
        Account a = new Account();
        a.display();
        // System.out.println(a.balance()); ⟶ Error.
        // a.balance = 10000; ⟶ Error.
    }
}
```

(iii) **protected**:- A protected member can be accessed within the same class, within the same package and in subclass of another package.

Ex:-
```
class Vehicle {
    protected int speed = 100;
}

class Car extends Vehicle {
    void display(){
        System.out.println(speed);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.display();
    }
}
```

(iv) **public** :- A public member can be accessed from anywhere.

Ex:-

```
class Calculator {
    public int add(int a, int b) {
        return a+b;
    }
}

public class Main {
    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
    }
}
```

Static members in Java

Static members are the members of a class that belong to the class itself, not to individual objects.

They are created only once and are shared by all objects of the class.

There are mainly 3 types of static members :-

(i) Static variable :- A static variable is shared among all objects.

Ex :-

```
class Student {
    static String college = "ABC college";
    String name;

    Student (String n){
      name = n;
    }
}

class Main {
    public static void main(String[] args) {
        Student s1 = new Student ("Rahul");
        Student s2 = new Student ("Aman");

        System.out.println(s1.college);
        System.out.println(s2.college);
    }
}
```

(ii) Static Method :- A static method belongs to the class and can be called without creating object.

Ex :-
```
class Calculator {
    static int square(int n) {
        return n*n;
    }
}
class Main {
    public static void main(String[] args) {
        System.out.println(Calculator.square(5));
    }
}
```

(iii) Static Block :- A static block is executed once when the class is loaded.

Ex :-
```
class Test {
    static {
        System.out.println("Static block executed");
    }

    public static void main(String[] args) {
        System.out.println("Main method executed");
    }
}
```

Output :- Static block executed
Main method executed.

Final variables in Java

* A final variable is a variable whose value cannot be changed once it has been assigned.

* The final keyword is used to create final variables.

* It must be initialized before use.

* By convention, final constants are written in uppercase.

```
final int x=10;
x= 20; // Error
```

Ex:- final variable with object.

```
class Student {
    final int rollNo= 101;

    void display(){
        System.out.println(rollNo);
    }
}

class Main {
    public static void main (String [] args){
        Student s= new Student();
        s.display();
    }
}
```

Inner Classes in Java

Ajanta
Page No.
Date

An inner class is a class that is declared inside another class. It is logically used to group classes and can access the members of its outer class.

Ex:-
```
class Outer {
    int x = 20;

    class Inner {
        void display() {
            System.out.println(x);
        }
    }
}

class Main {
    public static void main(String[] args) {
        Outer obj = new Outer();
        Outer.Inner in = obj.new Inner();

        in.display();
    }
}
```

Types of Inner Class:-

(i) Member Inner Class:- declared inside a class but outside methods.

(ii) Static Nested Class:- declared using static.

(iii) Local Inner Class:- declared inside a method.

(iv) Anonymous Inner class:- class without a name, created for single use.

Four main concept of oops

(i) Encapsulation:- Wrapping data (variable) and methods (functions) into a single unit (class) and restricting direct access to data.

(ii) Abstraction:- Hiding the internal implementation details and showing only the essential features of an object.

(iii) Inheritance:- A method where one class acquires the properties and methods of another class.

(iv) Polymorphism:- The ability of an object to take many forms, allowing the same method to behave differently in different classes.

Types:- Compile Time Polymorphism:- Method overloading

Run-Time Polymorphism:- Method overriding.

Command line Arguments in Java

* command line arguments are values passed to a Java program when it is run from the command prompt/Terminal.

* They are received by the main() method the String array args

Syntax:- public static void main(String[] args)

* Command line arguments are stored in strings, so no.s need conversion using Integer.parseInt().

Ex:- class Demo {

```
    public static void main (String [] args) {
        System.out.println("Name :" + args[0]);
        System.out.println("Age :" + args[1]);
    }
}
```

java Demo Ajay 20 ⟶ More
args[0] = Ajay
args[1] = 20

• Variable-length Argument (Varargs) in Java.

* variable-length arguments allow a method to accept any number of arguments.

They are represented using ...

Syntax:- returnType methodname (dataType... variable)

Ex:- class Demo {

```
    static int sum(int... numbers) {
        int s=0

        for (int n : numbers)
            s+=n;
        return s;
    }
    public static void main (String [] args) {

        System.out.println(sum(10, 20));
        "   "   "   (sum(10, 20, 30, 40));
```

Inheritance in Java

Inheritance is a OOPs concept in which one class acquires the properties and methods of another class.

Super class / Parent class / Base class

↓

Sub class / child class / Derived class.

Types of Inheritance

(i) Single Inheritance :- One child class inherits from one parent class.

Ex:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

(iii) Multilevel Inheritance:- A class inherits from another class which inherits from another class.

Ex:-

```
class Animal{
    void eat(){
        System.out.println("Eating");
    }
}
class Dog extends Animal{
    void bark(){
        System.out.println("Barking");
    }
}
class Puppy extends Dog{
    void play(){
        System.out.println("Playing");
    }
}

class Main{
    public static void main(String[] args){
        Puppy p = new Puppy();

        p.eat();
        p.bark();
        p.play();
    }
}
```

(iii) Hierarchical Inheritance :- Multiple child class inherit from the same parent class.

Ex:-

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Meowing");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();

        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}
```

(iv) Multiple Inheritance:- One class inherits multi...

Ajanta
Page No.
Date

NOTE:- Java does not support multiple inheritance usi... classes but it can be acheived using interfa...

Ex:

```
interface father{
    void work();
}

interface Mother{
    void cook();
}

class child implements father, Mother{
    public void work(){
        System.out.println("father works");
    }
    public void cook(){
        System.out.println("Mother's cooking");
    }
}

class Main{
    public static void main(String[] args){
        Child c= new Child();
        c.work();
        c.cook();
    }
}
```

(v) Hybrid Inheritance:- A combination of two or more types of inheritance.

Ajanta
Page No.
Date

* In Java, it can be implemented using classes and interfaces.

Ex:-
```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

interface Pet {
    void play();
}

class Puppy extends Dog implements Pet {
    public void play() {
        System.out.println("Playing");
    }
}
class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();

        p.eat();
        p.bark();
        p.play();
    }
}
```

super keyword in Java

Ajanta
Page No.
Date

The super keyword is used to refer to the immediate parent class of the current class.

It is mainly used in 3 ways:-

(i) Access Parent class variable:- When parent and child classes have variables with the same name, super is used to access the parent variable.

Ex:-

```
class Animal {
    String name = "Animal";
}

class Dog extends Animal {
    String name = "Dog";

    void show() {
        System.out.println(name);
        System.out.println(super.name);
    }
}

class Main {
    public static void main (String[] args) {
        Dog d = new dog();
        d.show();
    }
}
```

(ii) Call parent class Method :- super.MethodName() is used to call a method of the parent class.

Ex:-

```
class Animal {
    void sound() {
        System.out.println("Animals makes sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }

    void display() {
        super.sound();
        sound();
    }
}

class Main {
    public static void main(String[] args) {

        Dog d = new Dog();
        d.display();

    }
}
```

(iii) Call parent class Constructor

Ajanta
Page No.
Date

super() is used to call the constructor of the parent class.

Ex:-
```
class Animal {
    Animal() {
        System.out.println("Animal Constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog Constructor");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Protected Membrs in Java

It is a variable or method declared using the protected access modifier.

Protected membrs can be accessed:-

* Within the same class.
* By other classes in the same package.
* By subclasses even if they are in different package.

Calling order of Constructors in Java

Ajanta
Page No.
Date

* When inheritance is used, constructors are called in order from the parent class to the child class.

* C object created → Constructor A → Constructor B → Constructor C

Ex:-

```
class A {
    A() {
        System.out.println("A");
    }
}
class B {
    B() {
        system.out.println("B");
    }
}
class C {
    C() {
        system.out.println("C");
    }
}

class Main {
    public static void main(String[] args) {
        C obj = new C();
    }
}
```

Method overriding in Java

* It occurs when a subclass provides its own implementation to a method that is already defined in its superclass.

* In the child class must have the same name, parameters and compatible return type as the parent method.

* It provides runtime polymorphism.

* @override is an optional annotation that helps detect mistake.

Ex:-
```
class vehicle {
    void start () {
        System.out.println("vehicle starts");
    }
}
class Car extends vehicle {
    @override
    void start () {
        System.out.println("Car starts");
    }
}
class main {
    public static void main (String[] args) {
        vehicle v = new Car ();
        v.start (); // output:- Car starts.
    }
}
```

Object class in Java

The object class is the root class of Java. Every class in Java directly or indirectly inherits from object.

Methods of object class:- toString()
equals()
hashcode()
getclass()
clone()

Ex:-
```
class Student {
    public string tostring(){
        return "Student Object";
    }

    public static void main(String[] args){
        Student s= new Student();
        System.out.println(s.toString());
    }
}
```

Abstract class in Java

* An abstract class is a class declared using the abstract keyword.
* It is used as a base class and cannot be directly instantiated.
* We cannot create an object of an abstract class.
* A subclass must implement its abstract methods.

Ex:- abstract class shape {

```
    abstract void area();
    void display(){
        System.out.println("This is a shape");
    }
}

class Square extends Shape {
    void area(){
        System.out.println("Area = side x side");
    }
}

class Main {
    public static void main(String[] args){
        Square s = new Square();
        s.area();
        s.display();
    }
}
```

Abstract Method in Java

* An abstract method is a method that has only a declaration and no body.
* It must be declared inside an abstract class or interface.
* Must be implemented by the subclass.

Ex:-
```
abstract class Employee {
    abstract void work();
}

class developer extends Employee {
    void work() {
        System.out.println("Developer writes code");
    }
}

class Main {
    public static void main (String[] args) {
        Developer d = new Developer();
        d.work();
    }
}
```

final keyword with Inheritance in Java

The final keyword is used to restrict inheritance and modification. It can be used as with classes, methods and variables.

(i) final class :- A class declared as final cannot be inherited.

Ex:-
```
final class vehicle {
    void start() {
        System.out.println("vehicle starts");
    }
}

//Error.
class Car extends vehicle {
}
```

(ii) Final Method:- A final method cannot be overridden by a subclass.

Page No.
Date

Ex:-
```
class Bank {
    final void rates() {
        ---------
    }
}

class SBI extends Bank {

    //Error.
    void rates() {
    }
```

(iii) Final variable:- A final variable can be assigned only one.

Ex:-
```
class Student {
    final int marks = 90;

    void display() {
        System.out.println(marks);

        //Error.
        marks = 95;
    }
}
```